Practical 8: Implementation of Priority Scheduling (Preemptive & Non-Preemptive)

Author: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: Feburary 25, 2026

Topic: Implementation of scheduling algo: Priotity Scheduling

1. Theory: Priority Scheduling

Priority Scheduling assigns the CPU to the process with the highest priority (lowest number = highest priority). It can be:

- **Non-Preemptive**: Once a process starts, it runs to completion.
- **Preemptive**: If a new process arrives with a higher priority, the CPU is preempted.

.

2. C Implementation

Non-Preemptive Priority Scheduling

```
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, priority, waiting, turnaround, completed;
};

void inputProcesses(struct Process p[], int n) {
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival: ", i + 1);
    scanf("%d", &p[i].arrival);
    printf("P%d Burst: ", i + 1);
    scanf("%d", &p[i].burst);
    printf("P%d Priority: ", i + 1);
    scanf("%d", &p[i].priority);
    p[i].completed = 0;
  }
}

void calculatePriorityNP(struct Process p[], int n) {
  int completed = 0, time = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min_prio = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].priority <
min_prio) {
```

```
        min_prio = p[i].priority;
        idx = i;
      }
    }
    if (idx == -1) { time++; continue; }
    printf("P%d\t\t%d\t%d\n", p[idx].pid, time, time + p[idx].burst);
    p[idx].waiting = time - p[idx].arrival;
    time += p[idx].burst;
    p[idx].turnaround = p[idx].waiting + p[idx].burst;
    p[idx].completed = 1;
    completed++;
  }
}

void printTable(struct Process p[], int n) {
  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tWT\tTAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

int main() {
  struct Process p[MAX];
  int n;
  printf("Non-Preemptive Priority\nEnter number of processes: ");
  scanf("%d", &n);
  inputProcesses(p, n);
  calculatePriorityNP(p, n);
  printTable(p, n);
  return 0;
}
```

Preemptive Priority Scheduling

```
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, remaining, priority, waiting, turnaround,
completed;
};
```

```c
void inputProcesses(struct Process p[], int n) {
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival: ", i + 1);
    scanf("%d", &p[i].arrival);
    printf("P%d Burst: ", i + 1);
    scanf("%d", &p[i].burst);
    printf("P%d Priority: ", i + 1);
    scanf("%d", &p[i].priority);
    p[i].remaining = p[i].burst;
    p[i].completed = 0;
  }
}

void calculatePriorityP(struct Process p[], int n) {
  int completed = 0, time = 0, last_idx = -1, start_time = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min_prio = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].priority <
min_prio) {
        min_prio = p[i].priority;
        idx = i;
      }
    }
    if (idx != last_idx) {
      if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
      start_time = time;
      last_idx = idx;
    }
    if (idx == -1) { time++; continue; }
    p[idx].remaining--;
    time++;
    if (p[idx].remaining == 0) {
      p[idx].completed = 1;
      completed++;
      p[idx].turnaround = time - p[idx].arrival;
      p[idx].waiting = p[idx].turnaround - p[idx].burst;
    }
  }
  if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
}

void printTable(struct Process p[], int n) {
  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tWT\tTAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
```

```
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

int main() {
  struct Process p[MAX];
  int n;
  printf("Preemptive Priority\nEnter number of processes: ");
  scanf("%d", &n);
  inputProcesses(p, n);
  calculatePriorityP(p, n);
  printTable(p, n);
  return 0;
}
```

3. Output Screenshots

- **Non-Preemptive Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr8
❯ ./a.out
Non-Preemptive Priority
Enter number of processes: 5
P1 Arrival: 2
P1 Burst: 4
P1 Priority: 2
P2 Arrival: 2
P2 Burst: 3
P2 Priority: 3
P3 Arrival: 1
P3 Burst: 6
P3 Priority: 1
P4 Arrival: 3
P4 Burst: 1
P4 Priority: 5
P5 Arrival: 2
P5 Burst: 5
P5 Priority: 4

Order of Execution:
process_name    start   end
P3              1       7
P1              7       11
P2              11      14
P5              14      19
P4              19      20

PID     AT      BT      WT      TAT
P1      2       4       5       9
P2      2       3       9       12
P3      1       6       0       6
P4      3       1       16      17
P5      2       5       12      17

Average Waiting Time: 8.40
Average Turnaround Time: 12.20
```

- **Preemptive Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr8
❯ ./a.out
Preemptive Priority
Enter number of processes: 5
P1 Arrival: 2
P1 Burst: 4
P1 Priority: 2
P2 Arrival: 2
P2 Burst: 3
P2 Priority: 3
P3 Arrival: 1
P3 Burst: 6
P3 Priority: 1
P4 Arrival: 3
P4 Burst: 1
P4 Priority: 5
P5 Arrival: 2
P5 Burst: 5
P5 Priority: 4

Order of Execution:
process_name    start   end
P3              1       7
P1              7       11
P2              11      14
P5              14      19
P4              19      20

PID     AT      BT      WT      TAT
P1      2       4       5       9
P2      2       3       9       12
P3      1       6       0       6
P4      3       1       16      17
P5      2       5       12      17

Average Waiting Time: 8.40
Average Turnaround Time: 12.20
```